Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Lab Report 08:Write a program to compute FIRST of terminals and non-terminals of a given ContextFree Grammar (CFG)

Submitted by:

Name	Mohammad Hafizur Rahman Sakib
ID	0222210005101118
Section	C
Session	Fall 2025 (8th Semester)
Date of performance	04.04.2026
Submission Date	11.04.2026

Submitted to:
Nazma Akther
Assistant Professor, Department of CSE
Premier University,Chattogram

Remarks

Experiment Name

Write a program to compute FIRST of terminals and non-terminals of a given Context-Free Grammar (CFG).

Objective

To compute the FIRST set for all terminals and non-terminals in a given context-free grammar (CFG). The FIRST set of a symbol X is the set of terminals that appear as the first symbols in strings derived from X.

Algorithm

1. Start the program.
2. Read the number of production rules from the user. For each production, read the left-hand side (non-terminal) and the right-hand side (sequence of symbols), and store them in the grammar.
3. Identify all terminals and non-terminals from the grammar:
   - Any symbol appearing on the left-hand side of a production is a **non-terminal**.
   - Any symbol that is not a non-terminal is a **terminal** (including ε).
4. Initialize FIRST sets:
   - For each terminal a, set $\text{FIRST}(a) = \{a\}$, since a terminal derives only itself.
   - For each non-terminal A, initialize $\text{FIRST}(A) = \emptyset$ (empty set).
5. Apply the following fixed-point iteration — repeat until no FIRST set changes:
   - For each production rule $A \rightarrow X_1\ X_2\ X_3\ \cdots\ X_n$, process each symbol X_i from left to right:
     - **Case 1:** If X_1 is a terminal a (or ε), add a (or ε) directly to $\text{FIRST}(A)$ and stop processing this production.
     - **Case 2:** If X_i is a non-terminal B, add all symbols in $\text{FIRST}(B) - \{\varepsilon\}$ to $\text{FIRST}(A)$.
     - **Case 3:** If $\varepsilon \in \text{FIRST}(X_i)$, then X_i is nullable — continue to the next symbol X_{i+1} and repeat.
     - **Case 4:** If $\varepsilon \notin \text{FIRST}(X_i)$, stop processing this production — the remaining symbols cannot contribute to $\text{FIRST}(A)$.

- **Case 5:** If all symbols $X_1, X_2, \ldots, X_n$ are nullable (i.e., ε is in all their FIRST sets), then add ε to FIRST(A).

6. After the iteration converges (no more changes occur), the FIRST sets are complete.
7. Display the computed FIRST sets for all non-terminals.
8. End the program.

Input Grammar

$$
\begin{aligned}
E &\rightarrow T\ E' \\
E' &\rightarrow +\ T\ E' \mid \varepsilon \\
T &\rightarrow F\ T' \\
T' &\rightarrow *\ F\ T' \mid \varepsilon \\
F &\rightarrow (\ E\ ) \mid \textbf{id}
\end{aligned}
$$

Source Code

```cpp
#include <bits/stdc++.h>
using namespace std;

map<string, vector<vector<string>>> G;
map<string, set<string>> FIRST;

bool NT(string s){ return isupper(s[0]); }

vector<string> split(string s){
    vector<string> v; string w; stringstream ss(s);
    while(ss>>w) v.push_back(w);
    return v;
}
int main(){
    int n; string line,prod;
    cout<<"Enter number of productions: ";
    cin>>n; cin.ignore();

    cout<<"Enter productions:\n";
    while(n--){
        getline(cin,line);
        int p=line.find("->");
        string L=line.substr(0,p-1), R=line.substr(p+3);
        stringstream ss(R);
        while(getline(ss,prod,'|')) G[L].push_back(split(prod));
    }

    for(auto &g:G) FIRSTof(g.first);

    cout<<"\nFIRST sets:\n";
    for(auto &g:G){
        cout<<"FIRST("<<g.first<<") = { ";
        for(auto &x:FIRST[g.first]) cout<<x<<" ";
        cout<<"}\n";
    }
}
```

Figure 1: Source code for computing FIRST sets of a given grammar

Output Screenshot

Figure 2: Output showing computed FIRST sets

Discussion

This experiment demonstrates the implementation of the FIRST set computation algorithm for context-free grammars. The FIRST set is a fundamental concept in compiler construction, particularly used in the design of top-down parsers such as the LL(1) parser.

The program correctly handles all cases: productions starting with a terminal (directly added to the FIRST set), productions starting with a non-terminal (FIRST of that symbol minus ε is propagated), nullable non-terminals (algorithm looks ahead to the next symbol), and epsilon productions (added when all symbols in a production are nullable).